DoppelDash

Enterprise Suite · Full Architecture Blueprint

LMS · RMS · Stakeholder CRM · RBAC · Self-Hosted · OCR

Confidential · v1.0 · 2026

■ LMS	■ RMS	■ CRM	■ RBAC	■■ Self-Host	■ OCR
Leave tracking & workflows	3-tier reimbursement pipeline	Stakeholder intelligence OS	Dynamic role management	Your server + internet	Device camera business cards

What DoppelDash Actually Is

DoppelDash is not a SaaS product — it is a **self-hosted enterprise operating system** that your organisation runs on its own server, accessible to the internet via HTTPS, with no dependency on any third-party cloud for core data. Three tightly integrated modules — LMS, RMS, and the Stakeholder CRM — are unified under a single dynamic role-based access system, a shared audit log, and one deployment.

Self-Hosted Deployment Architecture

Your server · Your data · Live on the internet

■ Internet / Users
HTTPS via domain

■■ Cloudflare
DDoS · CDN · SSL termination

■ Nginx Reverse Proxy
Port 80/443 → internal services

■■ Next.js App (Docker)
SSR · API routes · frontend

■■ MongoDB (Docker)
Primary database · audit logs

■ MinIO (Docker)
Self-hosted S3 · files · receipts · docs

■ Redis (Docker)
Job queues · sessions · cache

■■ Your Physical Server
Ubuntu 22 LTS · Docker Compose

How Internet Access Works

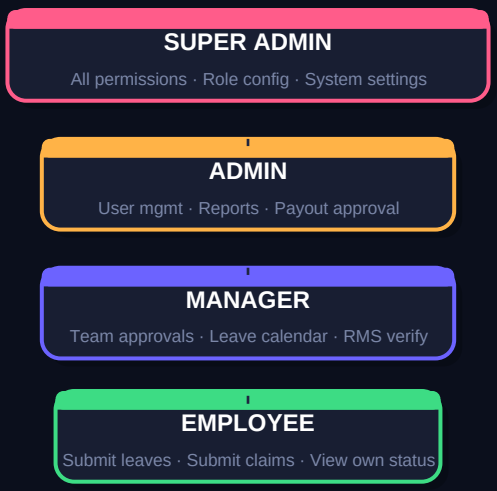
Your server sits in your office or data centre. Cloudflare acts as the security and CDN layer between the world and your machine — **no one connects directly to your server IP**. Nginx on your server receives the proxied HTTPS request and routes it to the correct Docker container. SSL certificates are handled automatically via Cloudflare (edge) or Let's Encrypt (origin).

Component	Technology	Why This Choice
OS	Ubuntu 22.04 LTS	5-year LTS, Docker-native, battle-tested
Containerisation	Docker + Docker Compose	One command spin-up, easy backups, portable
Reverse Proxy	Nginx	SSL termination, load balance, static serve
DNS / CDN / DDoS	Cloudflare (free tier)	Hides server IP, free SSL, DDoS shield
App Server	Next.js 15 (Node.js)	SSR + API routes in one container
Database	MongoDB 7 (Docker volume)	Flexible schema, audit-log-friendly
File Storage	MinIO (Docker)	S3-compatible, fully self-hosted
Job Queue	BullMQ + Redis (Docker)	Background jobs: email, OCR, greetings

Component	Technology	Why This Choice
Email	Resend API	Transactional email, no SMTP headaches
Backups	Daily mongodump + MinIO sync	Automated cron → external drive or NAS

Dynamic Role-Based Access Control (RBAC)

No hardcoded users — every permission is role-driven and configurable



Role Architecture

Role	LMS Permissions	RMS Permissions	CRM Permissions
Super Admin	Configure all leave types & quotas	View all · configure payout rules	Full access · encryption keys
Admin	View org-wide reports · override	Final payout · mark paid + proof	All stakeholders · bulk ops
Manager	Approve/reject team leaves	Verify claims · approve (no pay)	Department stakeholders
Employee	Submit · view own balance	Submit claims + receipts	Read-only own contacts

How Roles Are Created

Roles are stored in a **roles** collection in MongoDB. Each role has a permissions array of granular capability strings (e.g. *lms:approve*, *rms:payout*, *crm:encrypt_view*). The Super Admin can create new roles, clone existing ones, and assign them to users — all from the admin panel. There are **no hardcoded user IDs** anywhere in the codebase. Adding a new approval tier (e.g. a Finance role between Manager and Admin) requires zero code changes.

Collection	Key Fields	Notes
roles	_id, name, permissions[], createdBy, createdAt	Seeded with 4 defaults; fully editable
users	_id, name, email, roleId (FK), department, isActive	roleId references roles collection
permissions	key, label, module, description	Master list of all capability strings

■ Leave Management System (LMS)

Zero-manual-tracking time-off workflows with audit-grade documentation

Approval Workflow

Feature	Specification	Tech
Leave Categories	Casual · Medical · Earned — all configurable by Admin	MongoDB leave_types collection
Balance Allocation	Admin sets per-employee annual quotas per category	Admin panel UI + roles
Medical Proof Upload	Mandatory if Medical leave > 2 days · drag-and-drop zone	MinIO + browser-image-compression
File formats accepted	PDF · JPG · PNG · max 5 MB per file	Multer → MinIO
Manager Approval	Approve with note · Reject with mandatory reason	BullMQ job on action
Automated Email on Decision	Fires immediately via Resend to employee's email	Resend API + email template
Team Calendar Heatmap	Manager sees full department coverage before approving	React + date-fns
Monthly Reports	Downloadable per department — XLSX + PDF	SheetJS + jsPDF
Balance Deduction	Automatic on approval — not on submission	MongoDB transaction
Holiday Config	Admin configures public holidays — auto excluded	holidays collection

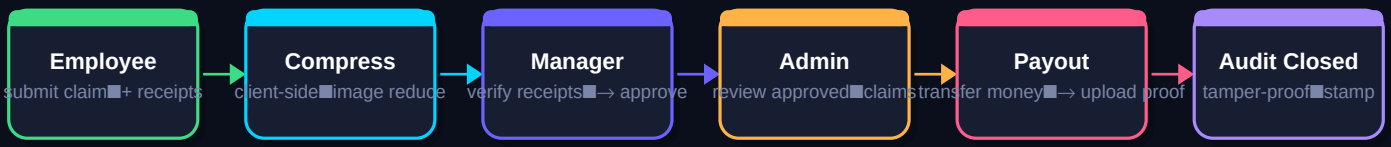
Database Schema — LMS

Collection	Key Fields
leave_requests	_id · employeeId · type · startDate · endDate · days · status · proofUrl · managerId · decisionAt · reason
leave_balances	_id · employeeId · year · casual{total,used} · medical{total,used} · earned{total,used}
leave_types	_id · name · requiresProofAfterDays · color · isActive

■ Reimbursement Management System (RMS)

3-tier financial pipeline — zero leaks, full audit trail

3-Tier Approval Pipeline



Feature	Specification	Tech
Expense Categories	Travel · Medical · Other — extensible by Admin	expense_types collection
Receipt Upload	Multi-image mandatory · compressed client-side before upload	browser-image-compression → MinIO
Compression Target	Each image < 500 KB before upload regardless of source size	client-side only, no server load
Manager Verification	Review claim + view receipts · Approve with note or Reject	Signed MinIO URLs (time-limited)
Manager Cannot Pay	Hard permission block — rms:payout not in Manager role	Middleware permission check
Admin Payout Step	Clicks Mark as Paid — MUST upload bank screenshot OR enter Txn ID, both required	Joi validation
Payment Proof Storage	Screenshot stored in MinIO · Txn ID in DB — both permanent	MinIO + MongoDB
Automated Email Alerts	Employee notified at Manager approval AND at payout	Resend — 2 triggers
Tamper-Proof Audit Log	Every action appended to audit_log — never updated/deleted	Append-only MongoDB collection
Financial Reports	Department breakdowns · monthly trends · XLSX + PDF export	SheetJS + jsPDF + Chart.js

Database Schema — RMS

Collection	Key Fields
reimbursements	_id · employeeId · category · amount · description · receiptUrls[] · status · managerId · managerNote · adminId · txnId · pay
audit_log	_id · entityType · entityId · action · actorId · actorRole · metadata{} · createdAt [APPEND ONLY]

■ Stakeholder CRM

Hierarchical intelligence platform — not a sales funnel

Core Focus

This CRM is not HubSpot. It tracks **stakeholder relationships, grievances, and interaction timelines** within a corporate hierarchy. It is security-first (field-level encryption on PII), automation-heavy (Outlook sync, AI drafting, job queues), and zero-data-entry via OCR business card scanning.

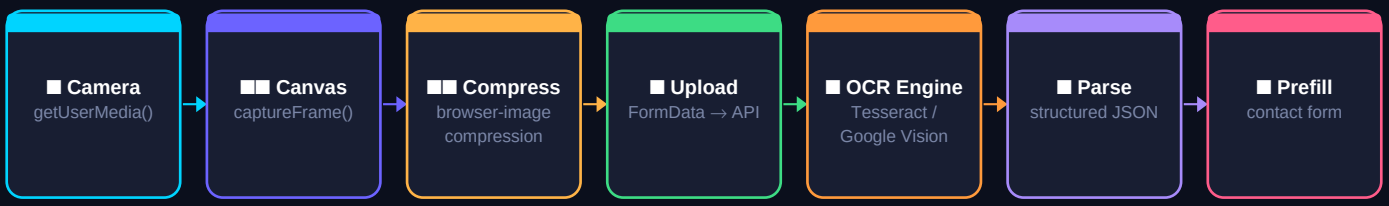
Feature	Specification	Tech
Stakeholder Profiles	Name · org · role · department · interaction history	MongoDB contacts collection
Hierarchical Tracking	Org tree — who reports to whom, escalation paths	Parent/child refs in schema
Grievance Routing	Log complaint → assign owner → track resolution	grievances collection + BullMQ
Interaction Timeline	Every email, call, note — auto or manual — chronological	interactions[] per contact
Field-Level Encryption	Phone, email, sensitive notes encrypted at app layer	Node.js crypto (AES-256-GCM)
Deduplication Engine	Fuzzy match on name+email+phone before insert	Fuse.js + levenshtein distance
Outlook / Email Sync	Microsoft Graph API pulls emails → auto-log to timeline	Graph API + BullMQ poll job
AI Message Generator	Context-aware email/follow-up draft from stakeholder data	OpenAI GPT-4o via API
Automated Greeting Jobs	Birthday · anniversary auto-emails via queue	BullMQ scheduled jobs + Resend
OCR Business Card Scanner	Camera → OCR → prefill contact form (see next page)	Device camera + Tesseract / Vision

Database Schema — CRM

Collection	Key Fields
contacts	_id · name · org · roleTitle · department · phone(enc) · email(enc) · parentId · tags[] · createdBy · dedupeHash
interactions	_id · contactId · type(email|call|note|meeting) · summary · source(manual|outlook|ai) · actorId · createdAt
grievances	_id · contactId · title · description · status · assignedTo · resolvedAt · escalationHistory[]
crm_email_queue	_id · contactId · type · scheduledAt · status · template · sentAt

■ OCR Business Card Scanner

Device camera → instant digital contact — zero manual typing



How It Works in the Browser

When a user clicks *Scan Business Card*, the app calls **navigator.mediaDevices.getUserMedia** to access the device camera — this works on phones, tablets, and laptops via any modern browser, with no app install required. The user points the camera at the card and taps Capture. A canvas element grabs the frame, **browser-image-compression** reduces it to under 500 KB, and it is sent to the Next.js API route. Server-side, **Tesseract.js** (self-hosted, free) performs OCR. If the self-hosted accuracy is insufficient for a particular card, the system falls back to **Google Vision API** (pay-per-use). The raw OCR text is parsed by a regex + GPT-4o prompt into structured JSON (name, org, phone, email, title) and returned to the frontend to prefill the form.

Step	What Happens	Tech	Self-Hosted?
Camera	getUserMedia() — rear cam preferred	Browser WebRTC API	■ Yes
Capture	Canvas.drawImage() on button tap	HTML5 Canvas	■ Yes
Compress	Reduce to <500 KB before upload	browser-image-compression	■ Yes
Upload	FormData POST to /api/ocr	Next.js API route	■ Yes
OCR Primary	Tesseract.js on server	Tesseract.js (Node)	■ Yes — free
OCR Fallback	Google Vision API if confidence < 80%	Google Cloud Vision	■■ Paid API
Parse	Regex + GPT prompt → structured JSON	OpenAI GPT-4o	■■ API call
Prefill	Form fields auto-filled, user confirms	React state	■ Yes
Dedup Check	Fuzzy match before final save	Fuse.js	■ Yes

Privacy Note on Camera Access

The browser will show a native permission prompt the first time. The camera feed is never stored — only the single captured frame is sent to the server, processed, and immediately discarded. No video is recorded or transmitted.

Full Technology Stack

Layer	Technology	Purpose
Framework	Next.js 15 (App Router)	SSR · API routes · file-based routing
UI	Tailwind CSS + ShadCN UI	Design system — consistent across modules
Database	MongoDB 7 + Mongoose	Primary store — schema-flexible
File Storage	MinIO (self-hosted S3)	Receipts · medical docs · payment proofs · OCR images
Job Queue	BullMQ + Redis	Email sends · OCR · greeting jobs · Outlook poll
Email	Resend API	All transactional emails — LMS + RMS alerts
Auth	NextAuth.js + JWT	Session management + RBAC middleware
Encryption	Node.js crypto — AES-256-GCM	Field-level PII encryption in CRM
OCR Primary	Tesseract.js (server-side)	Self-hosted business card OCR
OCR Fallback	Google Cloud Vision API	High-accuracy fallback
AI	OpenAI GPT-4o	Message drafting · OCR parsing · context
Outlook Sync	Microsoft Graph API	Email pull → stakeholder timeline
Image Compress	browser-image-compression	Client-side — before any upload
Deduplication	Fuse.js + custom levenshtein	CRM contact dedup engine
Reports	SheetJS (XLSX) + jsPDF	Financial + leave reports
Reverse Proxy	Nginx	SSL · routing · static files
CDN / DDoS	Cloudflare (free tier)	Security · performance · SSL edge
Containers	Docker + Docker Compose	Full stack in one compose file
Backups	mongodump cron + MinIO replication	Daily automated backups

Security Architecture

Concern	Solution
Authentication	NextAuth.js sessions + HTTP-only cookies — no localStorage tokens
Authorisation	Middleware checks roleId → permissions[] on every API route
PII Encryption	CRM phone/email encrypted with AES-256-GCM before MongoDB write
File Access	MinIO pre-signed URLs with 15-min expiry — no public buckets
Audit Log	Append-only audit_log collection — no UPDATE or DELETE allowed

Concern	Solution
Outlook Token Storage	Encrypted refresh tokens in DB — never exposed to frontend
Server Exposure	Cloudflare proxy hides origin IP — only 443 open to Cloudflare IPs
Dependency Updates	Dependabot + monthly review — no EOL packages in production

Build Roadmap

Phase	Timeline	Deliverables	Status
Phase 1	Wk 1–2	Project scaffold · Docker Compose · Nginx · MongoDB · MinIO · Auth	Foundation
Phase 2	Wk 3–4	RBAC system · role editor · user management · permission middleware	Identity
Phase 3	Wk 5–7	LMS — leave types · balance · submission · medical upload	Core LMS
Phase 4	Wk 8–9	LMS — manager approval · Resend alerts · calendar heatmap · reports	LMS Complete
Phase 5	Wk 10–12	RMS — claim submission · receipt compression · manager verify	Core RMS
Phase 6	Wk 13–14	RMS — admin payout + proof · audit log · financial reports	RMS Complete
Phase 7	Wk 15–17	CRM — stakeholder profiles · hierarchy · interaction timeline	Core CRM
Phase 8	Wk 18–19	CRM — field-level encryption · dedup engine · grievance routing	CRM Security
Phase 9	Wk 20–21	CRM — OCR camera scanner · Tesseract + Vision fallback	OCR
Phase 10	Wk 22–23	CRM — Outlook sync · AI message drafting · greeting job queue	Automation
Phase 11	Wk 24	Load testing · penetration test · backup validation · go-live	Launch

Key Integration Dependencies

Integration	Requires	Lead Time
Resend Email	API key — sign up at resend.com	Minutes
OpenAI GPT-4o	API key + billing account	Minutes
Google Vision OCR	GCP project + Vision API enabled + billing	1 day
Microsoft Graph API	Azure AD app registration — admin approval	3–7 days (IT dept)
Cloudflare	Domain DNS moved to Cloudflare nameservers	1 day propagation
SSL Certificate	Cloudflare edge cert (automatic)	Instant via Cloudflare
MinIO	Docker — no external account needed	Self-hosted, immediate

Summary

DoppelDash is a **fully self-hosted, internet-accessible enterprise suite** that your organisation owns completely — no vendor lock-in, no SaaS fees, no data leaving your server. The LMS protects company time. The RMS protects company money. The CRM protects stakeholder relationships. RBAC means any approval tier can be configured without a code change. OCR means business cards become digital contacts in under 10 seconds using the camera

already in your hand. Everything runs in Docker, so moving to a bigger server takes one command.